Name - Adarsh Singh
Superset Id - **7741812**
VIT Bhopal

Design principles & Patterns

Exercise 1:Implementing the Singleton Pattern

Code:

```
class Logger {

   private static Logger instance;


   // Private constructor to prevent instantiation from outside

   private Logger() {

       System.out.println("Logger instance created.");

   }


   // Public static method to get the single instance of Logger

   public static Logger getInstance() {

       if (instance == null) {

           instance = new Logger();

       }

       return instance;

   }


   // Method to log messages

   public void log(String message) {

       System.out.println("[LOG]: " + message);
```

```
    }
}


public class SingletonPatternExample {
    public static void main(String[] args) {
        System.out.println("=== Singleton Pattern Demo ===\n");



        Logger logger1 = Logger.getInstance();
        logger1.log("Application started.");

        Logger logger2 = Logger.getInstance();
        logger2.log("User logged in.");
        System.out.println("\n--- Verification ---");
        System.out.println("logger1: " + logger1.hashCode());
        System.out.println("logger2: " + logger2.hashCode());
        System.out.println("Are both instances the same? " + (logger1 ==
logger2));
        System.out.println("Are both instances the same object? " +
(logger1.equals(logger2)));
        Logger logger3 = Logger.getInstance();
        Logger logger4 = Logger.getInstance();

        System.out.println("\n--- Additional Verification ---");
        System.out.println("logger3: " + logger3.hashCode());
        System.out.println("logger4: " + logger4.hashCode());
        System.out.println("All four loggers are same instance? " +
            (logger1 == logger2 && logger2 == logger3 && logger3 ==
logger4));
```

```
    }

}
```

Output:

```
genc-dn5.0/week1/design-principle on  main [?] via  v23.0.2
❯ javac SingletonPatternExample.java && java SingletonPatternExample
=== Singleton Pattern Demo ===

Logger instance created.
[LOG]: Application started.
[LOG]: User logged in.

--- Verification ---
logger1: 2133927002
logger2: 2133927002
Are both instances the same? true
Are both instances the same object? true

--- Additional Verification ---
logger3: 2133927002
logger4: 2133927002
All four loggers are same instance? true
```

Exercise 2: Implementing the Factory Method Pattern

code :

```
interface Document {
    String getType();
    void open();
    void save();
    void close();
}
class WordDocument implements Document {
    @Override
    public String getType() {
        return "Word Document";
    }
    @Override
    public void open() {
        System.out.println("Opening Microsoft Word document...");
    }
    @Override
    public void save() {
        System.out.println("Saving Word document...");
    }
    @Override
    public void close() {
        System.out.println("Closing Word document...");
```

```
    }
}
class PdfDocument implements Document {
    @Override
    public String getType() {
        return "PDF Document";
    }
    @Override
    public void open() {
        System.out.println("Opening Adobe PDF document...");
    }
    @Override
    public void save() {
        System.out.println("Saving PDF document...");
    }
    @Override
    public void close() {
        System.out.println("Closing PDF document...");
    }
}
class ExcelDocument implements Document {
    @Override
    public String getType() {
        return "Excel Document";
    }
    @Override
    public void open() {
        System.out.println("Opening Microsoft Excel spreadsheet...");
    }
```

```
    @Override
    public void save() {
        System.out.println("Saving Excel spreadsheet...");
    }

    @Override
    public void close() {
        System.out.println("Closing Excel spreadsheet...");
    }
}

// Abstract DocumentFactory class
abstract class DocumentFactory {
    public abstract Document createDocument();
    // Template method that uses the factory method
    public void processDocument() {
        Document document = createDocument();
        System.out.println("Created: " + document.getType());
        document.open();
        document.save();
        document.close();
    }
}
class WordDocumentFactory extends DocumentFactory {
    @Override
    public Document createDocument() {
        return new WordDocument();
    }
}
```

```
class PdfDocumentFactory extends DocumentFactory {
    @Override
    public Document createDocument() {
        return new PdfDocument();
    }
}
class ExcelDocumentFactory extends DocumentFactory {
    @Override
    public Document createDocument() {
        return new ExcelDocument();
    }
}
public class FactoryMethodPatternExample {
    public static void main(String[] args) {
        System.out.println("=== Factory Method Pattern Demo ===\n");
        System.out.println("--- Creating Word Document ---");
        DocumentFactory wordFactory = new WordDocumentFactory();
        wordFactory.processDocument();

        System.out.println();
        System.out.println("--- Creating PDF Document ---");
        DocumentFactory pdfFactory = new PdfDocumentFactory();
        pdfFactory.processDocument();

        System.out.println();
        System.out.println("--- Creating Excel Document ---");
        DocumentFactory excelFactory = new ExcelDocumentFactory();
        excelFactory.processDocument();
        System.out.println();
```

```
        System.out.println("--- Direct Document Creation ---");

        Document wordDoc = new WordDocumentFactory().createDocument();

        Document pdfDoc = new PdfDocumentFactory().createDocument();

        Document excelDoc = new ExcelDocumentFactory().createDocument();



        System.out.println("Word Document Type: " + wordDoc.getType());

        System.out.println("PDF Document Type: " + pdfDoc.getType());

        System.out.println("Excel Document Type: " +
excelDoc.getType());

    }

}
```

Output:

```
genc-dn5.0/week1/design-principle on  main [?] via  v23.0.2
❯ javac FactoryMethodPatternExample.java && java FactoryMethodPatternExample
=== Factory Method Pattern Demo ===

--- Creating Word Document ---
Created: Word Document
Opening Microsoft Word document...
Saving Word document...
Closing Word document...

--- Creating PDF Document ---
Created: PDF Document
Opening Adobe PDF document...
Saving PDF document...
Closing PDF document...

--- Creating Excel Document ---
Created: Excel Document
Opening Microsoft Excel spreadsheet...
Saving Excel spreadsheet...
Closing Excel spreadsheet...

--- Direct Document Creation ---
Word Document Type: Word Document
PDF Document Type: PDF Document
Excel Document Type: Excel Document
```